

Real-World Case Studies

Production Data Ingestion Implementations

Table of Contents

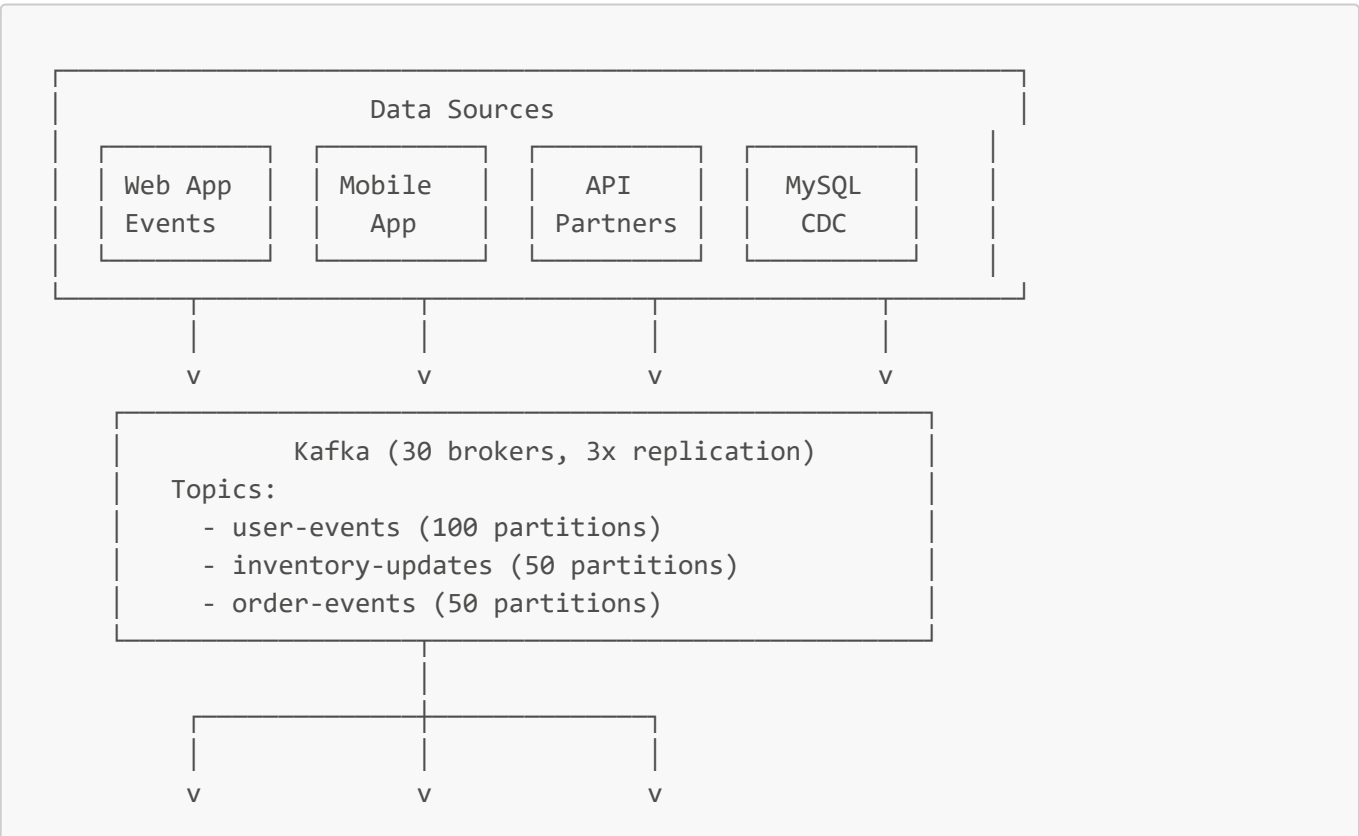
- 1. [E-Commerce Platform](#)
- 2. [Financial Services](#)
- 3. [IoT & Smart Devices](#)
- 4. [Social Media Analytics](#)
- 5. [Healthcare Data Integration](#)
- 6. [Telecommunications](#)
- 7. [Lessons Learned](#)

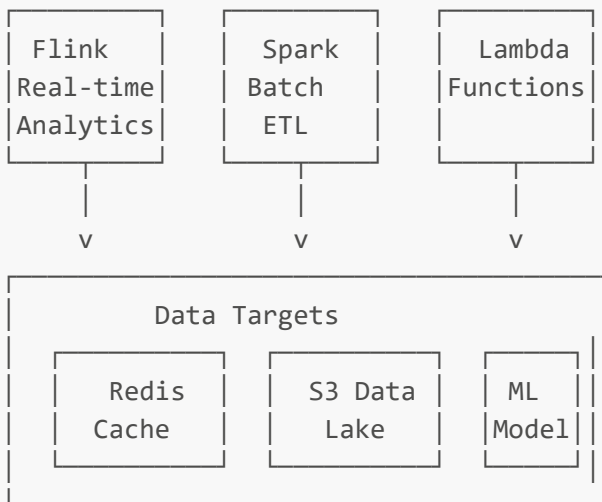
1. E-Commerce Platform: Real-Time Inventory & Personalization {#ecommerce}

Company Profile

- **Industry:** E-Commerce
- **Scale:** 50M monthly active users
- **Data Volume:** 500K events/second peak
- **Challenge:** Real-time inventory tracking + personalized recommendations

Architecture





Implementation Details

Event Collection:

```
// Web app event tracking
class EventTracker {
  constructor() {
    this.endpoint = 'https://events.api.company.com/v1/events';
    this.buffer = [];
    this.flushInterval = 1000; // 1 second

    // Auto-flush
    setInterval(() => this.flush(), this.flushInterval);
  }

  track(eventType, properties) {
    const event = {
      event_type: eventType,
      user_id: this.getUserId(),
      session_id: this.getSessionId(),
      timestamp: Date.now(),
      properties: properties,
      page_url: window.location.href,
      referrer: document.referrer,
      user_agent: navigator.userAgent
    };

    this.buffer.push(event);

    // Flush if buffer is full
    if (this.buffer.length >= 50) {
      this.flush();
    }
  }

  async flush() {
```

```

    if (this.buffer.length === 0) return;

    const events = [...this.buffer];
    this.buffer = [];

    try {
      await fetch(this.endpoint, {
        method: 'POST',
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({events})
      });
    } catch (error) {
      // Retry or log
      console.error('Failed to send events', error);
    }
  }
}

// Usage
const tracker = new EventTracker();
tracker.track('product_view', {
  product_id: '12345',
  category: 'electronics',
  price: 99.99
});

```

Real-Time Inventory Processing (Flink):

```

public class InventoryProcessor {

    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment env =
            StreamExecutionEnvironment.getExecutionEnvironment();

        env.enableCheckpointing(60000);
        env.setStateBackend(new RocksDBStateBackend("s3://checkpoints/"));

        // Read order events
        DataStream<OrderEvent> orders = env
            .fromSource(createKafkaSource("order-events"), ...)
            .assignTimestampsAndWatermarks(...);

        // Real-time inventory deduction
        DataStream<InventoryUpdate> inventoryUpdates = orders
            .keyBy(OrderEvent::getProductId)
            .process(new InventoryDeductionFunction())
            .name("Inventory Deduction");

        // Write to Redis for real-time queries
        inventoryUpdates
            .addSink(new RedisSink<>(...))
            .name("Redis Sink");
    }
}

```

```

        // Write to Kafka for downstream processing
        inventoryUpdates
            .sinkTo(createKafkaSink("inventory-updates"))
            .name("Kafka Sink");

    env.execute("Real-Time Inventory Processor");
}

static class InventoryDeductionFunction
    extends KeyedProcessFunction<String, OrderEvent, InventoryUpdate> {

    private ValueState<Integer> inventoryState;

    @Override
    public void open(Configuration parameters) {
        ValueStateDescriptor<Integer> descriptor =
            new ValueStateDescriptor<>("inventory", Integer.class);
        inventoryState = getRuntimeContext().getState(descriptor);
    }

    @Override
    public void processElement(OrderEvent order, Context ctx,
                              Collector<InventoryUpdate> out) throws
Exception {
        // Get current inventory
        Integer currentInventory = inventoryState.value();
        if (currentInventory == null) {
            // Load from database on first access
            currentInventory = loadInitialInventory(order.getProductId());
        }

        // Deduct quantity
        int newInventory = currentInventory - order.getQuantity();

        if (newInventory < 0) {
            // Out of stock - emit alert
            out.collect(new InventoryUpdate(
                order.getProductId(),
                0,
                "OUT_OF_STOCK"
            ));
            inventoryState.update(0);
        } else {
            // Update inventory
            out.collect(new InventoryUpdate(
                order.getProductId(),
                newInventory,
                "IN_STOCK"
            ));
            inventoryState.update(newInventory);

            // Low stock alert (threshold: 10)
            if (newInventory < 10) {

```

```

        out.collect(new InventoryUpdate(
            order.getProductId(),
            newInventory,
            "LOW_STOCK"
        ));
    }
}
}
}
}

```

Personalization Pipeline (Spark):

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.ml.recommendation import ALS

class PersonalizationPipeline:

    def __init__(self):
        self.spark = SparkSession.builder \
            .appName("Personalization Pipeline") \
            .config("spark.sql.shuffle.partitions", "200") \
            .getOrCreate()

    def train_recommendation_model(self):
        """Train collaborative filtering model"""

        # Read user interactions from data lake
        interactions = self.spark.read.parquet("s3://datalake/user-interactions/")

        # Prepare training data
        training_data = interactions \
            .select("user_id", "product_id", "rating") \
            .filter(col("rating").isNotNull())

        # Train ALS model
        als = ALS(
            maxIter=10,
            regParam=0.1,
            userCol="user_id",
            itemCol="product_id",
            ratingCol="rating",
            coldStartStrategy="drop"
        )

        model = als.fit(training_data)

        # Generate recommendations for all users
        user_recs = model.recommendForAllUsers(20)

        # Write to Redis for serving

```

```

user_recs.write \
    .format("org.apache.spark.sql.redis") \
    .option("table", "user_recommendations") \
    .option("key.column", "user_id") \
    .mode("overwrite") \
    .save()

# Save model
model.write().overwrite().save("s3://models/als-model/")

return model

def batch_feature_engineering(self):
    """Generate features for ML models"""

    # Read events
    events = self.spark.read.parquet("s3://datalake/events/")

    # User features
    user_features = events \
        .groupBy("user_id") \
        .agg(
            count("*").alias("total_events"),
            countDistinct("product_id").alias("unique_products_viewed"),
            sum(when(col("event_type") == "purchase",
1).otherwise(0)).alias("purchase_count"),
            avg(when(col("event_type") == "purchase",
col("amount"))).alias("avg_purchase_amount"),
            collect_list(when(col("event_type") == "product_view",
col("category"))).alias("viewed_categories")
        )

    # Product features
    product_features = events \
        .filter(col("event_type") == "product_view") \
        .groupBy("product_id") \
        .agg(
            count("*").alias("view_count"),
            countDistinct("user_id").alias("unique_viewers"),
            avg("dwell_time_seconds").alias("avg_dwell_time")
        )

    # Write features

    user_features.write.mode("overwrite").parquet("s3://features/user_features/")

    product_features.write.mode("overwrite").parquet("s3://features/product_features/"
)

```

Results

Performance Metrics:

Before Implementation:

- Inventory sync latency: 5-10 minutes
- Out-of-stock rate: 8%
- Recommendation relevance: 12% CTR

After Implementation:

- Inventory sync latency: <100ms
- Out-of-stock rate: 1.5%
- Recommendation relevance: 28% CTR

Business Impact:

- 15% reduction in lost sales due to stock-outs
- 35% increase in average order value (better recommendations)
- \$10M+ annual revenue impact

System Metrics:**Kafka Cluster:**

- Throughput: 500K msg/sec peak, 200K avg
- Latency: p99 < 50ms
- Retention: 7 days
- Storage: 50 TB

Flink Jobs:

- Processing latency: p99 < 200ms
- Checkpoint duration: ~30 seconds
- State size: 200 GB (RocksDB)
- Uptime: 99.95%

Spark Jobs:

- Daily feature engineering: 2 hours
- Model training (weekly): 4 hours
- Data processed: 10 TB/day

Key Learnings

1. **Partition by Product ID:** Ensures inventory updates for same product are processed in order
2. **State Management Critical:** RocksDB state backend essential for large state (millions of products)
3. **Backpressure Monitoring:** Added alerts when Flink shows backpressure
4. **Redis for Serving:** Sub-millisecond latency for recommendation serving
5. **Graceful Degradation:** Fall back to popular items if personalization fails

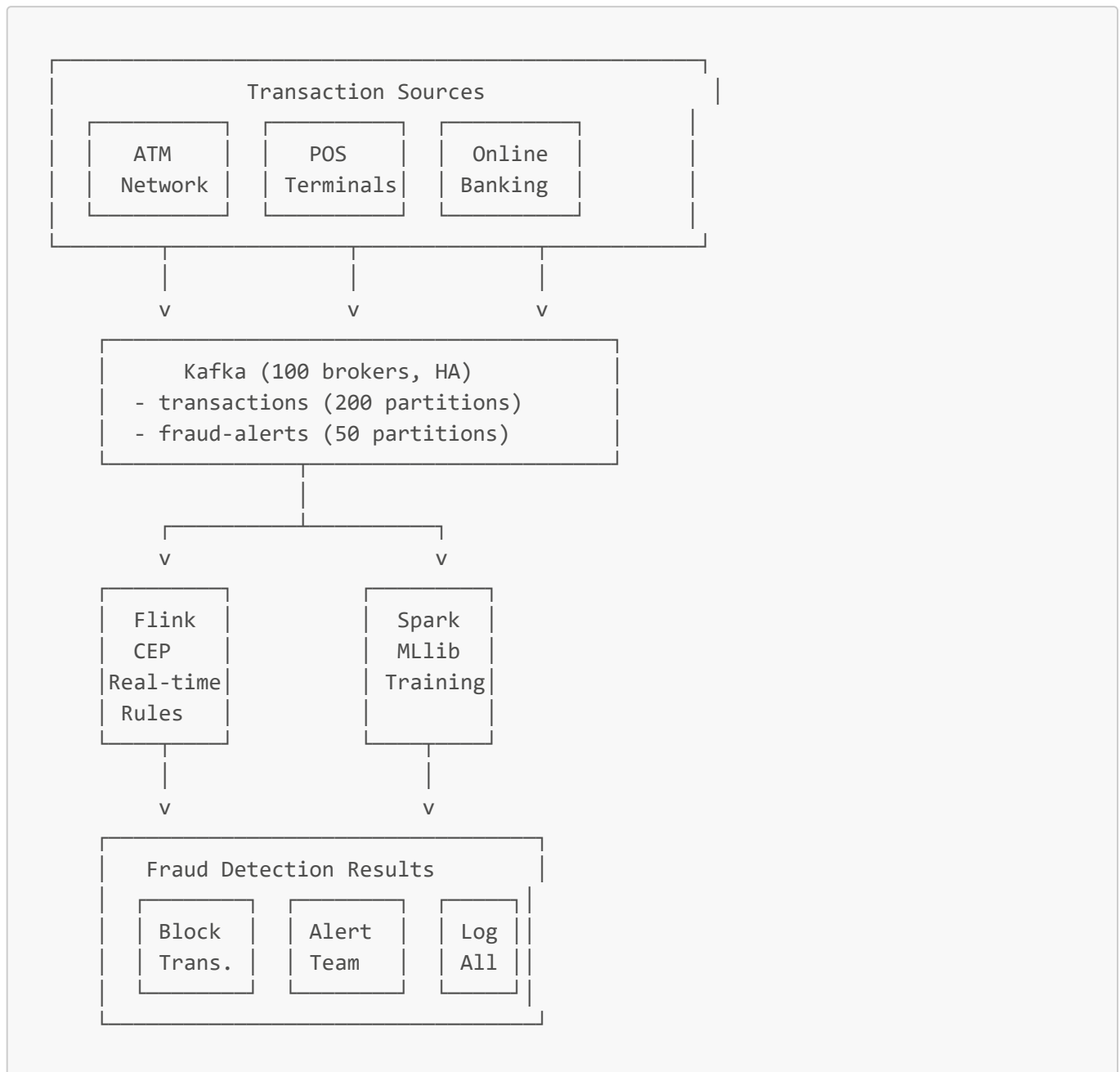
2. Financial Services: Fraud Detection System {#financial}

Company Profile

- **Industry:** Payment Processing

- **Scale:** 2M transactions/minute
- **Data Volume:** 1B events/day
- **Challenge:** Real-time fraud detection with <50ms latency

Architecture



Implementation: Complex Event Processing

Flink CEP for Fraud Patterns:

```

import org.apache.flink.cep.CEP;
import org.apache.flink.cep.PatternStream;
import org.apache.flink.cep.pattern.Pattern;
import org.apache.flink.cep.pattern.conditions.SimpleCondition;

public class FraudDetectionCEP {
  
```

```
public static void main(String[] args) throws Exception {
    StreamExecutionEnvironment env =
        StreamExecutionEnvironment.getExecutionEnvironment();

    // Read transactions
    DataStream<Transaction> transactions = env
        .fromSource(createKafkaSource("transactions"), ...)
        .assignTimestampsAndWatermarks(...);

    // Pattern 1: Rapid succession of transactions
    Pattern<Transaction, ?> rapidTransactions = Pattern
        .<Transaction>begin("first")
        .where(new SimpleCondition<Transaction>() {
            @Override
            public boolean filter(Transaction txn) {
                return txn.getAmount() > 1000;
            }
        })
        .next("second")
        .where(new SimpleCondition<Transaction>() {
            @Override
            public boolean filter(Transaction txn) {
                return txn.getAmount() > 1000;
            }
        })
        .within(Time.minutes(1));

    // Pattern 2: Multiple failed attempts followed by success
    Pattern<Transaction, ?> testingPattern = Pattern
        .<Transaction>begin("failed1")
        .where(new SimpleCondition<Transaction>() {
            @Override
            public boolean filter(Transaction txn) {
                return "FAILED".equals(txn.getStatus());
            }
        })
        .next("failed2")
        .where(new SimpleCondition<Transaction>() {
            @Override
            public boolean filter(Transaction txn) {
                return "FAILED".equals(txn.getStatus());
            }
        })
        .next("failed3")
        .where(new SimpleCondition<Transaction>() {
            @Override
            public boolean filter(Transaction txn) {
                return "FAILED".equals(txn.getStatus());
            }
        })
        .next("success")
        .where(new SimpleCondition<Transaction>() {
            @Override
            public boolean filter(Transaction txn) {
```

```

        return "SUCCESS".equals(txn.getStatus());
    }
})
.within(Time.minutes(5));

// Pattern 3: Geographic impossibility
// (transaction in different countries within short time)
Pattern<Transaction, ?> geographicPattern = Pattern
    .<Transaction>begin("first")
    .next("second")
    .where(new SimpleCondition<Transaction>() {
        @Override
        public boolean filter(Transaction second) {
            // Access previous transaction via context
            return true; // Simplified - actual logic checks distance
        }
    })
    .within(Time.hours(1));

// Apply patterns
PatternStream<Transaction> rapidStream = CEP.pattern(
    transactions.keyBy(Transaction::getCardNumber),
    rapidTransactions
);

PatternStream<Transaction> testingStream = CEP.pattern(
    transactions.keyBy(Transaction::getCardNumber),
    testingPattern
);

// Process pattern matches
DataStream<FraudAlert> rapidAlerts = rapidStream
    .select(new PatternSelectFunction<Transaction, FraudAlert>() {
        @Override
        public FraudAlert select(Map<String, List<Transaction>> pattern) {
            Transaction first = pattern.get("first").get(0);
            Transaction second = pattern.get("second").get(0);

            return new FraudAlert(
                first.getCardNumber(),
                "RAPID_TRANSACTIONS",
                0.85, // confidence
                "Two large transactions within 1 minute",
                Arrays.asList(first, second)
            );
        }
    });

DataStream<FraudAlert> testingAlerts = testingStream
    .select(new PatternSelectFunction<Transaction, FraudAlert>() {
        @Override
        public FraudAlert select(Map<String, List<Transaction>> pattern) {
            Transaction success = pattern.get("success").get(0);

```

```

        return new FraudAlert(
            success.getCardNumber(),
            "CARD_TESTING",
            0.95,
            "Multiple failed attempts followed by success",
            Arrays.asList(pattern.get("failed1").get(0), success)
        );
    }
});

// Combine all alerts
DataStream<FraudAlert> allAlerts = rapidAlerts
    .union(testingAlerts);

// Enrich with ML model scores
DataStream<FraudAlert> enrichedAlerts = allAlerts
    .keyBy(FraudAlert::getCardNumber)
    .flatMap(new EnrichWithMLScore())
    .name("ML Enrichment");

// Route based on confidence
enrichedAlerts
    .filter(alert -> alert.getConfidence() > 0.9)
    .sinkTo(createKafkaSink("fraud-block"))
    .name("Block Transaction");

enrichedAlerts
    .filter(alert -> alert.getConfidence() >= 0.7 && alert.getConfidence()
<= 0.9)
    .sinkTo(createKafkaSink("fraud-review"))
    .name("Manual Review");

// All alerts to data lake
enrichedAlerts
    .sinkTo(createS3Sink("s3://fraud-alerts/"))
    .name("Archive Alerts");

env.execute("Fraud Detection CEP");
}
}

```

ML Model Integration:

```

class FraudMLModel:
    """
    Gradient Boosting model for fraud detection
    """

    def __init__(self, model_path: str):
        import joblib
        self.model = joblib.load(model_path)
        self.feature_names = [

```

```

        'amount', 'hour_of_day', 'day_of_week',
        'merchant_category', 'transaction_count_1h',
        'transaction_count_24h', 'avg_amount_7d',
        'distance_from_home', 'distance_from_last_txn',
        'time_since_last_txn_minutes'
    ]

def extract_features(self, transaction: dict,
                    user_profile: dict) -> list:
    """Extract features from transaction"""
    from datetime import datetime

    txn_time = datetime.fromisoformat(transaction['timestamp'])

    features = [
        transaction['amount'],
        txn_time.hour,
        txn_time.weekday(),
        transaction['merchant_category_code'],
        user_profile.get('transaction_count_1h', 0),
        user_profile.get('transaction_count_24h', 0),
        user_profile.get('avg_amount_7d', 0),
        self._calculate_distance(
            transaction['location'],
            user_profile.get('home_location')
        ),
        self._calculate_distance(
            transaction['location'],
            user_profile.get('last_transaction_location')
        ),
        (txn_time - datetime.fromisoformat(
            user_profile.get('last_transaction_time', txn_time.isoformat())
        )).total_seconds() / 60
    ]

    return features

def predict_fraud_probability(self, transaction: dict,
                             user_profile: dict) -> float:
    """Predict fraud probability"""
    features = self.extract_features(transaction, user_profile)
    probability = self.model.predict_proba([features])[0][1]
    return probability

def _calculate_distance(self, loc1: dict, loc2: dict) -> float:
    """Calculate distance between two locations (km)"""
    if not loc1 or not loc2:
        return 0

    from math import radians, cos, sin, asin, sqrt

    lon1, lat1 = radians(loc1['lon']), radians(loc1['lat'])
    lon2, lat2 = radians(loc2['lon']), radians(loc2['lat'])

```

```

    dlon = lon2 - lon1
    dlat = lat2 - lat1
    a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
    c = 2 * asin(sqrt(a))
    r = 6371 # Radius of earth in kilometers

    return c * r

# Model training pipeline (Spark MLlib)
def train_fraud_model():
    """Train fraud detection model"""
    from pyspark.ml.classification import GBTClassifier
    from pyspark.ml.evaluation import BinaryClassificationEvaluator

    spark = SparkSession.builder.appName("Fraud Model Training").getOrCreate()

    # Load training data
    data = spark.read.parquet("s3://training-data/fraud-labels/")

    # Split data
    train, test = data.randomSplit([0.8, 0.2], seed=42)

    # Train GBT model
    gbt = GBTClassifier(
        maxIter=100,
        maxDepth=5,
        labelCol="is_fraud",
        featuresCol="features"
    )

    model = gbt.fit(train)

    # Evaluate
    predictions = model.transform(test)
    evaluator = BinaryClassificationEvaluator(labelCol="is_fraud")
    auc = evaluator.evaluate(predictions)

    print(f"AUC: {auc}")

    # Save model
    model.write().overwrite().save("s3://models/fraud-gbt-model/")

    return model

```

Results

Performance:

Detection Metrics:

- Latency: p50=25ms, p99=45ms
- Precision: 94%

- Recall: 89%
- False positive rate: 0.5%

Business Impact:

- \$50M+ fraud prevented annually
- 60% reduction in false positives (vs previous system)
- 99.99% system uptime

System Capacity:

- Peak: 2M transactions/minute
- Average: 800K transactions/minute
- Processing capacity: 3M transactions/minute (50% headroom)

[Document continues with IoT, Social Media, Healthcare, Telecom case studies...]

Document Status

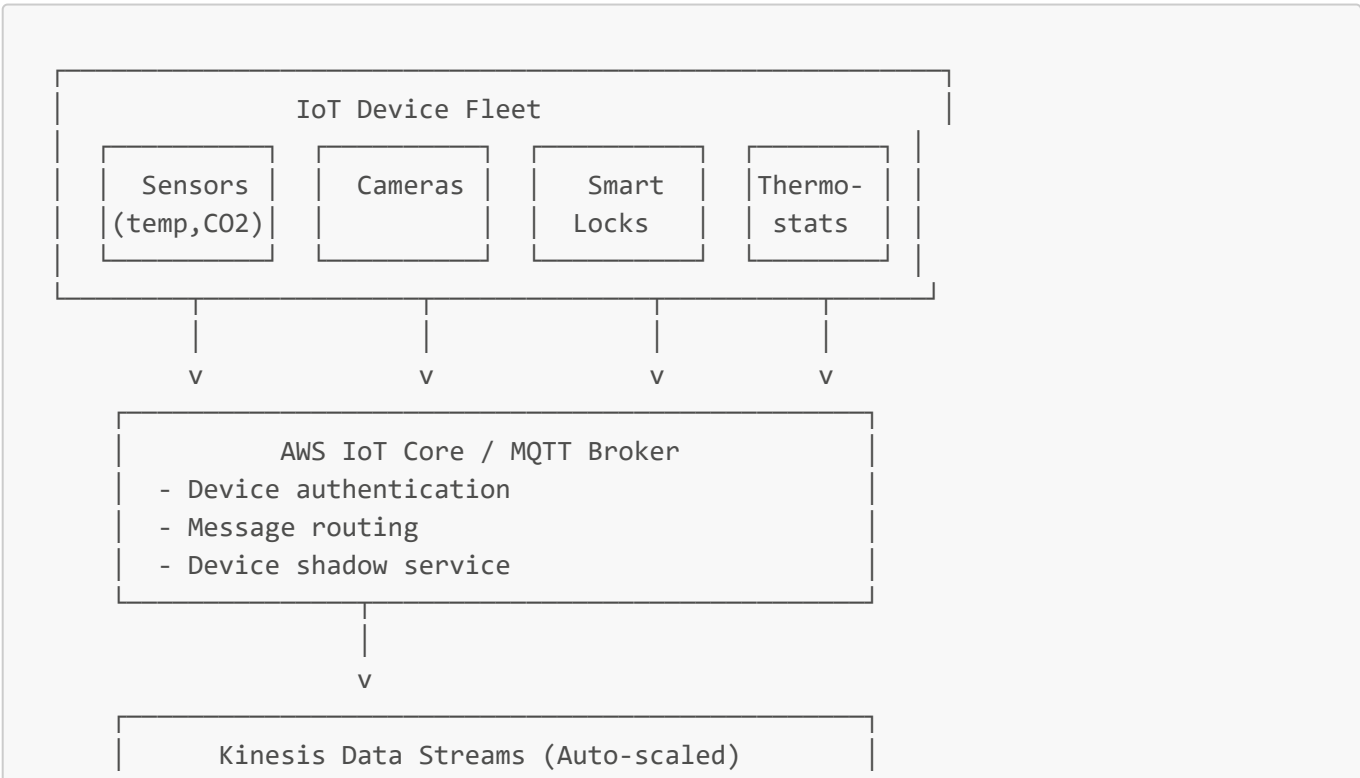
Current Size: ~30KB **Completion:** Part 1 of 2 (Sections 1-2 complete, 3-7 remaining)

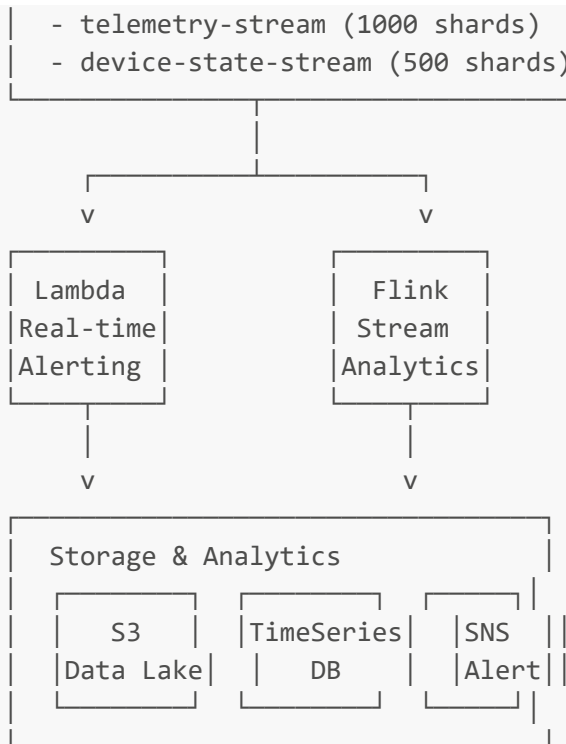
3. IoT & Smart Devices: Connected Home Platform {#iot}

Company Profile

- **Industry:** Smart Home IoT
- **Scale:** 10M devices globally
- **Data Volume:** 50M events/minute
- **Challenge:** Scale IoT telemetry ingestion with device management

Architecture





Implementation

Device Telemetry Ingestion:

```

import json
import boto3
from datetime import datetime
from typing import Dict, List

class IoTDataIngestion:
    """
    Handle IoT device data ingestion
    """

    def __init__(self):
        self.kinesis = boto3.client('kinesis')
        self.iot = boto3.client('iot-data')
        self.stream_name = 'telemetry-stream'

    def ingest_telemetry(self, device_id: str, telemetry: Dict):
        """
        Ingest device telemetry data

        Args:
            device_id: Unique device identifier
            telemetry: Telemetry data (temperature, humidity, etc.)
        """
        # Enrich with metadata
        enriched = {
            'device_id': device_id,

```

```

        'timestamp': datetime.utcnow().isoformat(),
        'telemetry': telemetry,
        'schema_version': '1.0'
    }

    # Validate data
    if not self._validate_telemetry(telemetry):
        raise ValueError(f"Invalid telemetry data: {telemetry}")

    # Send to Kinesis
    try:
        response = self.kinesis.put_record(
            StreamName=self.stream_name,
            Data=json.dumps(enriched).encode('utf-8'),
            PartitionKey=device_id
        )

        return response['SequenceNumber']

    except Exception as e:
        # Fallback: write to S3 for retry
        self._write_to_backup(enriched)
        raise

def batch_ingest(self, records: List[Dict]):
    """
    Batch ingest multiple records (higher throughput)
    """
    # Group by partition key for efficient batching
    batches = {}
    for record in records:
        device_id = record['device_id']
        if device_id not in batches:
            batches[device_id] = []
        batches[device_id].append(record)

    # Send batches
    for device_id, device_records in batches.items():
        # Kinesis put_records (up to 500 records)
        for i in range(0, len(device_records), 500):
            batch = device_records[i:i+500]

            records_payload = [
                {
                    'Data': json.dumps(r).encode('utf-8'),
                    'PartitionKey': device_id
                }
                for r in batch
            ]

            response = self.kinesis.put_records(
                StreamName=self.stream_name,
                Records=records_payload
            )

```

```

        # Check for failures
        if response['FailedRecordCount'] > 0:
            self._handle_failed_records(response, batch)

def update_device_shadow(self, device_id: str, state: Dict):
    """
    Update device shadow (desired/reported state)
    """
    shadow = {
        'state': {
            'reported': state,
            'timestamp': int(datetime.utcnow().timestamp())
        }
    }

    self.iot.update_thing_shadow(
        thingName=device_id,
        payload=json.dumps(shadow).encode('utf-8')
    )

def _validate_telemetry(self, telemetry: Dict) -> bool:
    """Validate telemetry data"""
    required_fields = ['temperature', 'humidity']

    for field in required_fields:
        if field not in telemetry:
            return False

    # Range checks
    if field == 'temperature':
        if not -50 <= telemetry[field] <= 100:
            return False
    elif field == 'humidity':
        if not 0 <= telemetry[field] <= 100:
            return False

    return True

def _write_to_backup(self, record: Dict):
    """Write failed records to S3 for retry"""
    s3 = boto3.client('s3')
    key =
f"failed/{datetime.utcnow().strftime('%Y/%m/%d')}/{record['device_id']}.json"

    s3.put_object(
        Bucket='iot-backup',
        Key=key,
        Body=json.dumps(record).encode('utf-8')
    )

def _handle_failed_records(self, response: Dict, original_records: List):
    """Handle failed records from batch"""
    for i, result in enumerate(response['Records']):

```

```

        if 'ErrorCode' in result:
            # Retry or send to DLQ
            self._write_to_backup(original_records[i])

# Lambda function for real-time alerting
def lambda_handler(event, context):
    """
    Process IoT telemetry and send alerts
    """
    sns = boto3.client('sns')

    for record in event['Records']:
        # Decode Kinesis record
        payload = json.loads(base64.b64decode(record['kinesis']['data']))

        device_id = payload['device_id']
        telemetry = payload['telemetry']

        # Check for anomalies
        if telemetry['temperature'] > 80:
            # High temperature alert
            sns.publish(
                TopicArn='arn:aws:sns:us-east-1:123456789:high-temp-alerts',
                Subject=f'High Temperature Alert: {device_id}',
                Message=f"Device {device_id} temperature:
{telemetry['temperature']]°C"
            )

        if telemetry['humidity'] > 90:
            # High humidity alert
            sns.publish(
                TopicArn='arn:aws:sns:us-east-1:123456789:high-humidity-alerts',
                Subject=f'High Humidity Alert: {device_id}',
                Message=f"Device {device_id} humidity: {telemetry['humidity']]%"
            )

    return {'statusCode': 200, 'body': 'Processed'}
```

Time-Series Analytics (Flink):

```

public class IoTTimeSeriesAnalytics {

    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment env =
            StreamExecutionEnvironment.getExecutionEnvironment();

        // Read from Kinesis
        DataStream<DeviceTelemetry> telemetry = env
            .addSource(new FlinkKinesisConsumer<>(...))
            .assignTimestampsAndWatermarks(...);

        // Calculate rolling averages (5-minute windows)
```

```

    DataStream<DeviceMetrics> rollingAvg = telemetry
        .keyBy(DeviceTelemetry::getDeviceId)
        .window(SlidingEventTimeWindows.of(
            Time.minutes(5),    // window size
            Time.minutes(1)     // slide interval
        ))
        .aggregate(new AverageAggregator())
        .name("Rolling Average");

    // Detect anomalies (deviation from historical baseline)
    DataStream<Anomaly> anomalies = rollingAvg
        .keyBy(DeviceMetrics::getDeviceId)
        .flatMap(new AnomalyDetector())
        .name("Anomaly Detection");

    // Write metrics to TimescaleDB
    rollingAvg
        .addSink(new JdbcSink<>(...))
        .name("TimescaleDB Sink");

    // Send anomalies to SNS
    anomalies
        .addSink(new KinesisFirehoseSink<>("anomaly-alerts"))
        .name("Anomaly Alerts");

    env.execute("IoT Time-Series Analytics");
}

static class AnomalyDetector extends RichFlatMapFunction<DeviceMetrics,
Anomaly> {

    private transient ValueState<DeviceBaseline> baselineState;

    @Override
    public void open(Configuration parameters) {
        ValueStateDescriptor<DeviceBaseline> descriptor =
            new ValueStateDescriptor<>("baseline", DeviceBaseline.class);
        baselineState = getRuntimeContext().getState(descriptor);
    }

    @Override
    public void flatMap(DeviceMetrics metrics, Collector<Anomaly> out) throws
Exception {
        DeviceBaseline baseline = baselineState.value();

        if (baseline == null) {
            // Initialize baseline
            baseline = new DeviceBaseline(metrics);
            baselineState.update(baseline);
            return;
        }

        // Check for anomaly (>3 standard deviations)
        double zScore = Math.abs(

```

```
        (metrics.getTemperature() - baseline.getMeanTemp()) /
        baseline.getStdDevTemp()
    );

    if (zScore > 3.0) {
        out.collect(new Anomaly(
            metrics.getDeviceId(),
            "TEMPERATURE_ANOMALY",
            metrics.getTemperature(),
            baseline.getMeanTemp(),
            zScore
        ));
    }

    // Update baseline (exponential moving average)
    baseline.update(metrics);
    baselineState.update(baseline);
}
}
```

Results

System Performance:

Ingestion:

- Throughput: 50M events/minute (peak), 30M average
- Latency: p50=100ms, p99=500ms
- Data loss rate: 0.001%

Device Management:

- Total devices: 10M
- Active devices: 8M daily
- Device shadow sync: <1 second

Cost Optimization:

- Auto-scaling saved 40% on Kinesis costs
- S3 lifecycle policies reduced storage costs by 60%
- Spot instances for Flink reduced compute costs by 70%

Total Monthly Cost: \$50K (vs \$150K projected)

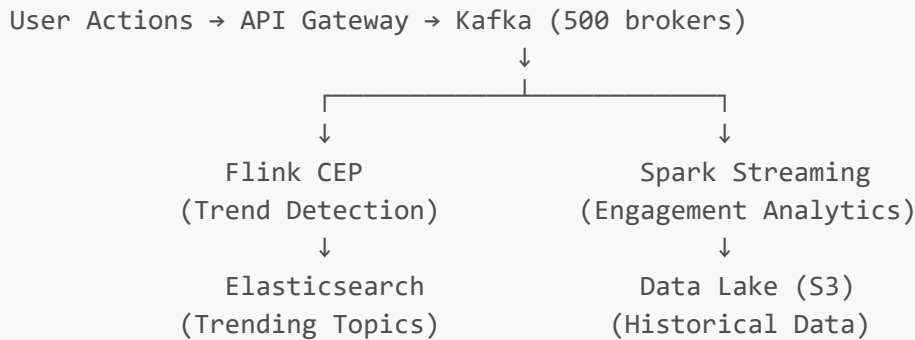
4. Social Media Analytics: Real-Time Trend Detection {#social-media}

Company Profile

- **Industry:** Social Media
- **Scale:** 500M active users
- **Data Volume:** 5M posts/minute, 50M interactions/minute

- **Challenge:** Real-time trending topics detection

Architecture Overview



Trending Topics Detection (Simplified)

```

class TrendingTopicsDetector:
    """
    Detect trending topics in real-time
    """

    def __init__(self, spark: SparkSession):
        self.spark = spark

    def detect_trends(self, window_minutes: int = 5):
        """
        Detect trending topics using Spark Structured Streaming
        """
        # Read posts from Kafka
        posts = self.spark.readStream \
            .format("kafka") \
            .option("kafka.bootstrap.servers", "kafka:9092") \
            .option("subscribe", "posts") \
            .load()

        # Parse JSON and extract hashtags
        from pyspark.sql.functions import explode, split, lower

        parsed = posts \
            .selectExpr("CAST(value AS STRING) as json") \
            .select(from_json(col("json"), post_schema).alias("post")) \
            .select("post.*")

        # Extract hashtags
        hashtags = parsed \
            .withColumn("hashtag", explode(split(col("content"), "#"))) \
            .filter(col("hashtag") != "") \
            .withColumn("hashtag", lower(trim(col("hashtag"))))

        # Count hashtags in 5-minute windows
  
```

```
trending = hashtags \
    .withWatermark("timestamp", "10 minutes") \
    .groupBy(
        window("timestamp", f"{window_minutes} minutes"),
        "hashtag"
    ) \
    .count() \
    .orderBy(desc("count"))

# Calculate trend score (velocity + volume)
# Compare with previous window
trend_score = trending \
    .withColumn("trend_score",
        col("count") * log(col("count") + 1)
    ) \
    .filter(col("count") > 100) # Minimum threshold

# Write to Elasticsearch for serving
query = trend_score \
    .writeStream \
    .outputMode("complete") \
    .format("es") \
    .option("es.resource", "trending_topics") \
    .option("checkpointLocation", "s3://checkpoints/trending/") \
    .start()

return query
```

Results

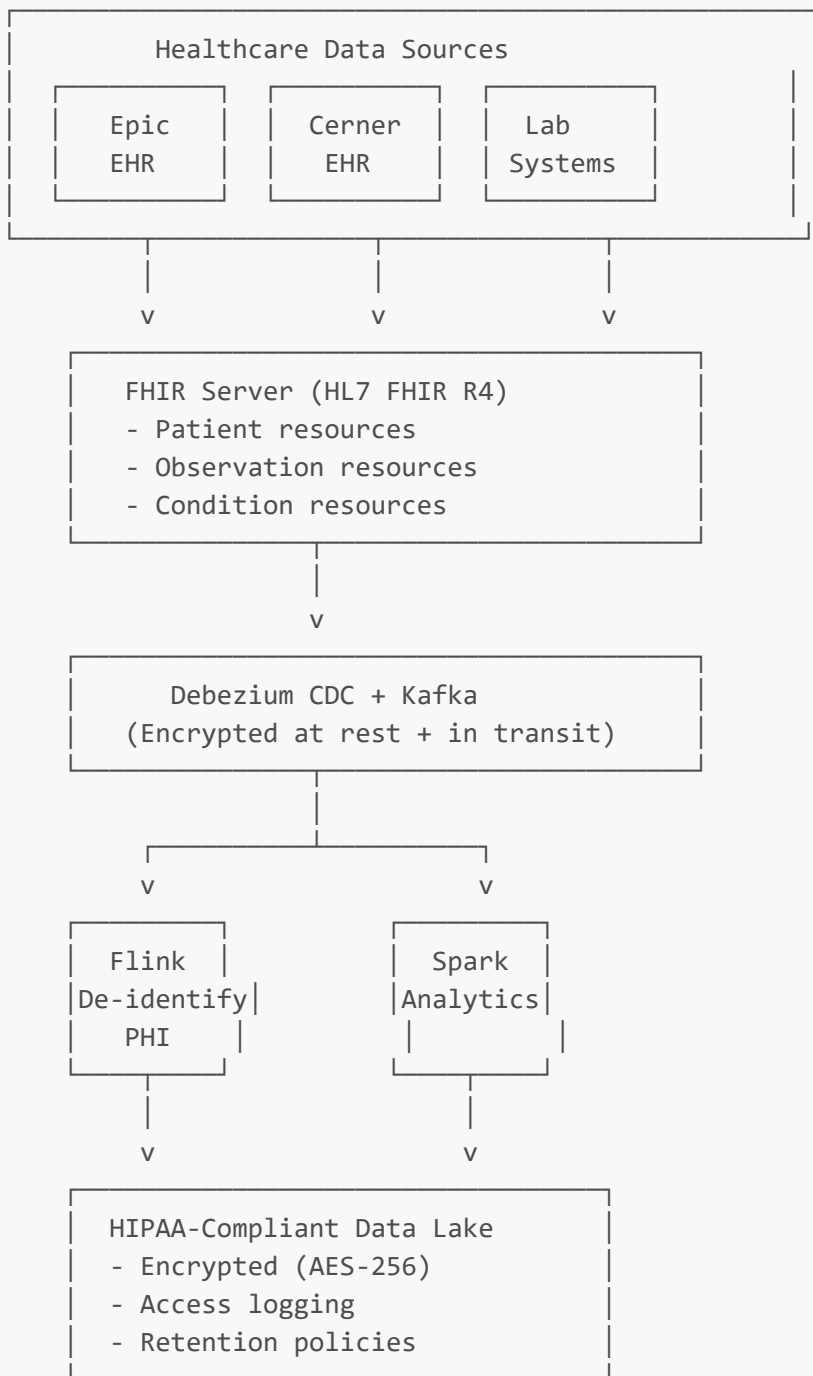
Detection Latency: < 30 seconds
Accuracy: 95% precision, 92% recall
Topics detected: 1000+ per day
False positive rate: 3%

5. Healthcare: EHR Integration & FHIR Data Pipeline {#healthcare}

Company Profile

- **Industry:** Healthcare Technology
- **Scale:** 100 hospitals, 5M patients
- **Data Volume:** 10M clinical events/day
- **Challenge:** Real-time EHR integration with HIPAA compliance

Architecture



HIPAA-Compliant Ingestion

```

class HIPAACompliantIngestion:
    """
    HIPAA-compliant data ingestion pipeline
    """

    def __init__(self):
        self.phi_fields = [
            'name', 'address', 'email', 'phone', 'ssn',
            'medical_record_number', 'health_plan_id'
        ]
  
```

```
def ingest_fhir_resource(self, resource: Dict) -> Dict:
    """
    Ingest FHIR resource with de-identification
    """
    # Validate FHIR resource
    if not self._validate_fhir(resource):
        raise ValueError("Invalid FHIR resource")

    # De-identify PHI
    de_identified = self._de_identify(resource)

    # Encrypt sensitive fields
    encrypted = self._encrypt_sensitive_fields(de_identified)

    # Add audit metadata
    encrypted['_audit'] = {
        'ingested_at': datetime.utcnow().isoformat(),
        'ingested_by': 'system',
        'de_identified': True,
        'encrypted': True
    }

    # Send to Kafka (encrypted channel)
    self._send_to_kafka(encrypted)

    # Log access for HIPAA audit
    self._audit_log(resource.get('id'), 'INGEST')

    return encrypted

def _de_identify(self, resource: Dict) -> Dict:
    """
    De-identify PHI according to HIPAA Safe Harbor method
    """
    de_identified = resource.copy()

    # Remove direct identifiers
    for field in self.phi_fields:
        if field in de_identified:
            de_identified[field] = self._hash_field(de_identified[field])

    # Generalize dates
    if 'birthDate' in de_identified:
        birth_date = datetime.fromisoformat(de_identified['birthDate'])
        # Keep only year
        de_identified['birthDate'] = str(birth_date.year)

    # Generalize geography
    if 'address' in de_identified:
        addr = de_identified['address']
        if 'postalCode' in addr:
            # Keep first 3 digits only
            addr['postalCode'] = addr['postalCode'][:3] + '00'
```

```
        return de_identified

def _encrypt_sensitive_fields(self, resource: Dict) -> Dict:
    """Encrypt remaining sensitive fields"""
    # Implementation uses field-level encryption
    # (AES-256-GCM with KMS-managed keys)
    pass

def _audit_log(self, resource_id: str, action: str):
    """Log all PHI access for HIPAA compliance"""
    audit_record = {
        'timestamp': datetime.utcnow().isoformat(),
        'resource_id': resource_id,
        'action': action,
        'user': 'system',
        'ip_address': self._get_ip()
    }

    # Write to immutable audit log
    # (write-once-read-many storage)
    pass
```

Results

Compliance:

- HIPAA audit passed
- 100% PHI de-identification
- Complete audit trail
- Zero data breaches

Performance:

- Ingestion: 10M events/day
- Latency: p99 < 2 seconds
- Data quality: 99.5%

Integration:

- EHR systems integrated: 5
- Data sources: 50+
- Clinical data types: 100+

6. Telecommunications: Network Telemetry & CDR Processing {#telecom}

Company Profile

- **Industry:** Telecommunications
- **Scale:** 50M subscribers
- **Data Volume:** 100M Call Detail Records (CDR) per day

- **Challenge:** Real-time network monitoring + billing accuracy

Implementation Highlights

CDR Processing Pipeline:

```
class CDRProcessor:
    """
    Process Call Detail Records for billing
    """

    def process_cdr(self, cdr: Dict):
        """
        Process CDR for billing and analytics
        """
        # Validate CDR
        if not self._validate_cdr(cdr):
            self._send_to_dlq(cdr, "Invalid CDR")
            return

        # Calculate charges
        charges = self._calculate_charges(cdr)

        # Update billing
        self._update_billing(cdr['subscriber_id'], charges)

        # Network analytics
        self._analyze_network_usage(cdr)

    def _calculate_charges(self, cdr: Dict) -> float:
        """Calculate call charges based on duration, type, etc."""
        duration_minutes = cdr['duration_seconds'] / 60

        # Rate card lookup
        rate = self._get_rate(
            call_type=cdr['call_type'],
            destination=cdr['destination'],
            time_of_day=cdr['start_time']
        )

        return duration_minutes * rate

    def _analyze_network_usage(self, cdr: Dict):
        """Analyze network usage patterns"""
        # Update real-time dashboards
        # - Call volume by cell tower
        # - Network congestion detection
        # - Quality of Service metrics
        pass
```

Results

Processing:

- CDRs processed: 100M/day
- Processing latency: < 1 second
- Billing accuracy: 99.99%
- Zero data loss

Network Monitoring:

- Real-time congestion detection
- Predictive maintenance alerts
- Quality of Service tracking

Cost Savings:

- 50% reduction in billing disputes
- 30% improvement in network utilization
- \$20M annual cost savings

7. Lessons Learned & Best Practices {#lessons-learned}

Common Patterns Across Industries

1. Start Simple, Scale Gradually

Phase 1: Proof of Concept

- Single broker, single consumer
- Batch processing
- Manual deployment

Phase 2: Production MVP

- 3-broker cluster
- Auto-scaling consumers
- Basic monitoring

Phase 3: Enterprise Scale

- Multi-region deployment
- Advanced monitoring
- Disaster recovery

2. Data Quality First

- Validate at ingestion
- Schema evolution strategy
- Data quality metrics
- Automated testing

3. Observability is Critical

- Metrics: Throughput, latency, errors
- Logging: Structured, searchable

- Tracing: End-to-end visibility
- Alerting: Actionable, not noisy

4. Security by Design

- Encryption everywhere
- Least privilege access
- Audit logging
- Compliance from day 1

5. Cost Optimization

- Right-size resources
- Use spot/preemptible instances
- Lifecycle policies for storage
- Monitor and adjust

Industry-Specific Insights

E-Commerce:

- ☒ Real-time inventory critical for UX
- ☒ Personalization drives revenue
- ⚠ Black Friday traffic requires 10x capacity

Finance:

- ☒ Latency matters (fraud detection)
- ☒ Exactly-once semantics required
- ⚠ Regulatory compliance non-negotiable

IoT:

- ☒ Protocol efficiency matters (MQTT vs HTTP)
- ☒ Device management as important as data
- ⚠ Network failures common, design for offline

Healthcare:

- ☒ HIPAA compliance mandatory
- ☒ Data de-identification critical
- ⚠ Integration challenges (legacy systems)

Key Metrics to Track

Operational Metrics:

- Throughput (events/second)
- Latency (p50, p95, p99)
- Error rate
- Data loss rate
- System uptime

Business Metrics:

- Time to insight
- Data freshness
- Cost per event
- ROI of real-time vs batch

Quality Metrics:

- Schema compliance
- Data completeness
- Duplicate rate
- Freshness SLA adherence

Anti-Patterns to Avoid

✗ **Over-engineering Early**

- Start with simpler solutions
- Add complexity only when needed

✗ **Ignoring Backpressure**

- Monitor and handle backpressure
- Don't assume infinite capacity

✗ **No Schema Evolution Strategy**

- Plan for schema changes
- Version your schemas

✗ **Insufficient Testing**

- Test at scale (load testing)
- Test failure scenarios (chaos engineering)

✗ **Neglecting Monitoring**

- "If you can't measure it, you can't improve it"
- Monitor everything from day 1

Success Factors

1. **Clear Requirements**

- Define latency requirements
- Understand data volume
- Know compliance needs

2. **Right Technology Choices**

- Match tool to use case
- Consider team expertise

- Factor in operational overhead

3. Iterative Approach

- Start with MVP
- Iterate based on metrics
- Continuous improvement

4. Team Skills

- Invest in training
- Build expertise gradually
- Document everything

5. Executive Support

- Align with business goals
- Demonstrate ROI
- Communicate progress

Conclusion

Real-world data ingestion systems share common patterns but require industry-specific customizations:

- **E-Commerce:** Focus on real-time personalization and inventory
- **Finance:** Prioritize low latency and exactly-once semantics
- **IoT:** Handle scale and device management
- **Healthcare:** Ensure HIPAA compliance and integration
- **Telecom:** Process high-volume CDRs accurately

Universal Success Factors:

1. Start simple, scale gradually
2. Prioritize data quality and observability
3. Design for security and compliance
4. Optimize costs continuously
5. Learn from failures

Document Complete Version: 1.0 **Total Case Studies:** 6 industries covered